VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMACINIŲ SISTEMŲ INŽINERIJOS STUDIJŲ PROGRAMA

Vienmatis optimizavimas

Laboratorinio darbo ataskaita

Atliko: Adomas Lukoševičius
VU el. p.: adomas.lukosevicius@mif.stud.vu.lt

Vertino: prof. dr. Julius Žilinskas

Vilnius
2026

UŽDUOTIS

1. SUPROGRAMUOKITE VIENMAČIO OPTIMIZAVIMO INTERVALO DALIJIMO PUSIAU, AUKSINIO PJŪVIO IR NIUTONO METODO ALGORITMUS

Naudojant *Golang* programavimo kalbą, implementavau intervalo dalijimo pusiau(žr. pav. 1), auksinio pjūvio (žr. pav. 2) ir Niutono metodo algoritmus (žr. pav. 3).

1.1. Intervalo dalijimo pusiau algoritmas

```go
func intervaloDalijimoPusiauAlgo(f tiksloFunkcija, l, r, epsilon float64, maxIter int) (xMin, fMin float64, iteracijuSk, funkcijosSkaiciavimuSk int, err error) {

    funkcijosSkaiciavimuSk = 0

    xm := (l + r) / 2

    for i := 0; i < maxIter; i++ {

        if (r - l) <= epsilon {
            x := (l + r) / 2
            return x, f(x), i, funkcijosSkaiciavimuSk + 1, nil
        }

        L := r - l

        x1 := l + L/4
        x2 := r - L/4

        if f(x1) < f(xm) {
            r = xm
            xm = x1

        } else if f(x2) < f(xm) {
            l = xm
            xm = x2

        } else {
            l = x1
            r = x2

        }
        funkcijosSkaiciavimuSk += 2

    }

    x := (l + r) / 2

    return x, f(x), maxIter, funkcijosSkaiciavimuSk + 1, fmt.Errorf("pasiektas maksimalus iteracijų skaičius.")

}
```

pav. 1 Intervalo dalijimo pusiau algoritmo kodas

1.2. Auksinio pjūvio algoritmas

```go
func auksinioPjuvioAlgo(f tiksloFunkcija, l, r, epsilon float64, maxIter int) (xMin, fMin float64, iteracijuNr, funkcijosSkaiciavimuSk int, err error) {

    funkcijosSkaiciavimuSk = 0

    tau := (math.Sqrt(5.0) - 1.0) / 2.0 // ~0.618

    x1 := r - tau*(r-l)
    x2 := l + tau*(r-l)

    for i := 0; i < maxIter; i++ {

        if (r - l) <= epsilon {
            x := (l + r) / 2
            return x, f(x), i, funkcijosSkaiciavimuSk + 1, nil
        }

        if f(x2) < f(x1) {
            l = x1
            x1 = x2
            x2 = l + tau*(r-l)

        } else {
            r = x2
            x2 = x1
            x1 = r - tau*(r-l)

        }
        funkcijosSkaiciavimuSk += 2
    }

    x := (l + r) / 2
    return x, f(x), maxIter, funkcijosSkaiciavimuSk + 1, fmt.Errorf("pasiektas maksimalus iteracijų skaičius.")
}
```

pav. 2 Auksinio pjūvio algoritmo kodas

1.3. Niutono metodo algoritmas

```go
func niutonoAlgo(f, df, d2f tiksloFunkcija, x0, epsilon float64, maxIter int) (xMin, fMin float64, iteracijuNr, funkcijosSkaiciavimuSk int, err error) {

    funkcijosSkaiciavimuSk = 0

    x := x0

    for i := 0; i < maxIter; i++ {

        if math.Abs(df(x)) <= epsilon {
            return x, f(x), i, funkcijosSkaiciavimuSk, nil
        }

        xNext := x - df(x)/d2f(x)
        funkcijosSkaiciavimuSk += 2

        if math.Abs(xNext-x) <= epsilon {
            x = xNext
            return x, f(x), i + 1, funkcijosSkaiciavimuSk, nil
        }
        x = xNext
    }

    return x, f(x), maxIter, funkcijosSkaiciavimuSk, fmt.Errorf("pasiektas maksimalus iteracijų skaičius.")
}
```

pav. 3 Niutono metodo algoritmo koda

2. APRAŠYKITE TIKSLO FUNKCIJĄ $F(X) = (X^2-A)^2/B-1$, ČIA A IR B – STUDENTO KNYGELĖS NUMERIO "2*1**AB" SKAITMENYS.

Pagal studento kodą, nustačiau A ir B reikšmes 3 ir 9 (žr. pav. 4).

```
// mano studento skaitmenys
var A float64 = 3
var B float64 = 9
```

pav. 4 A ir B reikšmės

Duotąją funkciją ir jos pirmosios bei antrosios eilės išvestines perrašiau *Golang* kalboje bendrąja forma (t.y. ne specifinėms A ir B reikšmėms) (žr. pav. 5)

```
// Tikslo funkcija + jos 1osios ir 2osios eilės isvestines
func funkcija(x float64) (y float64) {
    return math.Pow(x*x-A, 2) / (B - 1)
}

func isvestine_1(x float64) (y float64) {
    return (x * 4 * (x*x - A)) / (B - 1)
}

func isvestine_2(x float64) (y float64) {
    return (x*x*12 - A*4) / (B - 1)
}
```

pav. 5 Tikslo funkcija, 1-osios ir 2-osios eilės išvestinės

3. PALYGINKITE REZULTATUS: GAUTI SPRENDINIAI, RASTAS FUNKCIJOS MINIMUMO ĮVERTIS, ATLIKTŲ ŽINGSNIŲ IR FUNKCIJŲ SKAIČIAVIMŲ SKAIČIUS.

Naudojantis ankstesniais algoritmais minimizavau duotąją funkciją ir atlikau algoritmų efektyvumo tyrimą (žr. pav. 6)

```
**Intervalo Dalijimo Algoritmas**
xMin: 1.7320671081542969
fMin: 4.0e-10
iteraciju: 17
funkcijos skaiciavimu: 35

**Auksinio Pjuvio Algoritmas**
xMin: 1.7320263454721452
fMin: 9.0e-10
iteraciju: 24
funkcijos skaiciavimu: 25

**Niutono Algoritmas**
xMin: 1.732053083184218
fMin: 7.8e-12
iteraciju: 6
funkcijos skaiciavimu: 13
```

pav. 6 Tyrimo terminalo išvestis

3.1. Išvados

Atlikus tyrimą, palyginau gautus rezultatus. Svarbu paminėti, kad tikroji *xMin* vertė, su kuria bus lyginamas algoritmų efektyvumas buvo $\sqrt{3}$.

3.1.1. Intervalo dalijimo pusiau algoritmas

- Algoritmas atliko 17 iteracijų ir 35 kartus apskaičiavo *f(x)*.
- Gavo galutinę vertę 1,63 x 10^{-5} paklaida nuo tikrosios
- Lyginant su kitais:
    - Antroje vietoje pagal tikslumą, šiek tiek efektyviau už auksinio pjūvio algoritmą.
    - Iš visų algoritmų daugiausia kartų turėjo apskaičiuoti *f(x)* reikšmę (iš viso 35, ~2 funkcijų / iteraciją)

3.1.2. Auksinio pjūvio algoritmas

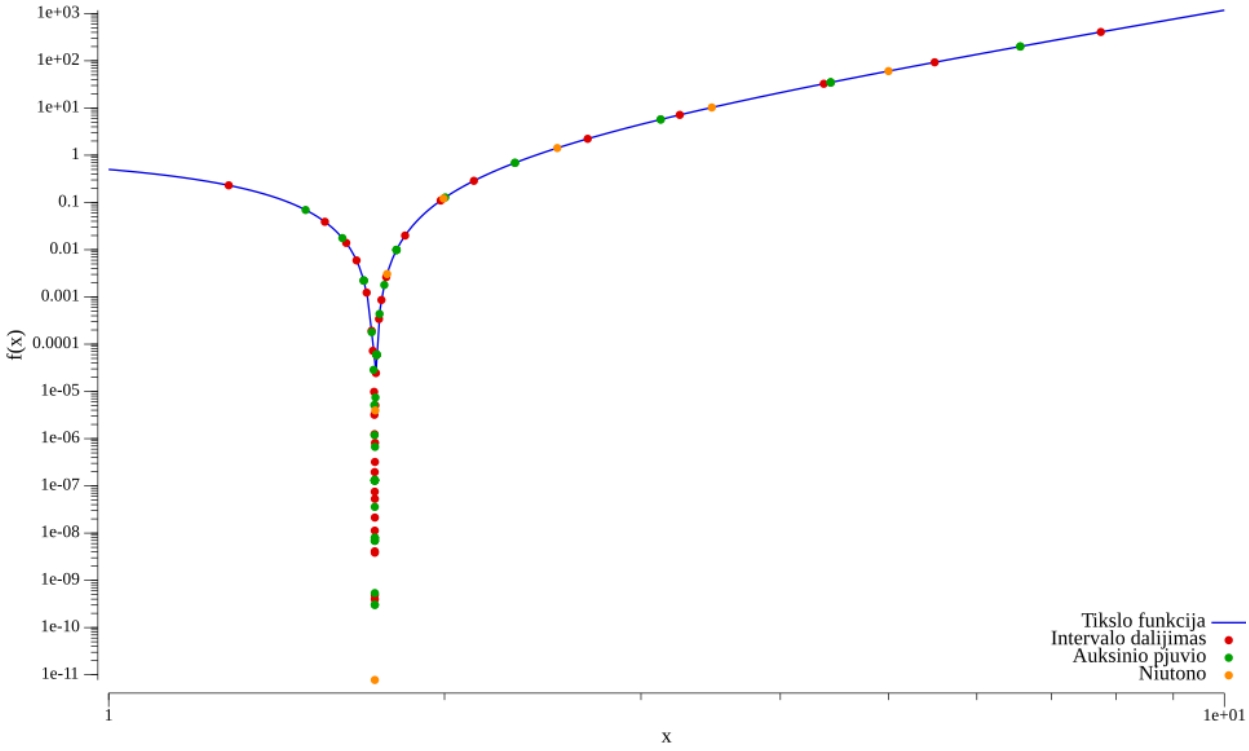
- Algoritmas atliko 24 iteracijų ir 25 kartus apskaičiavo *f(x)*.
- Gavo galutinę vertę 2,45 x 10^{-5} paklaida nuo tikrosios
- Lyginant su kitais:
    - Žemiausias tikslumas iš visų algoritmų
    - Efektyviausiai išnaudojo *f(x)* skaičiavimus (~1 funkcija / iteraciją)
    - Geriausias algoritmas neskaičiuojant išvestinių

3.1.3. Niutono metodo algoritmas

- Algoritmas atliko 6 iteracijas ir 13 kartų apskaičiavo *f(x)*.
- Gavo galutinę vertę 2,28 x 10^{-6} paklaida nuo tikrosios
- Lyginant su kitais:
    - Gavo daug tikslesnę vertę, negu kiti algoritmai
    - Greičiausiai gavo rezultatą (tik 6 iteracijos)
    - Vienintelis algoritmas, naudojęs išvestines (realus efektyvumas priklausytų nuo išvestinių apskaičiavimo kainos)

4. VIZUALIZUOKITE TIKSLO FUNKCIJĄ IR BANDYMO TAŠKUS.

Naudojantis *gonum plot* biblioteka, nubrėžiau grafiką, rodantį tikslo funkciją, apibrėžtą ribose [1, 10] bei visų 3 algoritmų bandymo taškus (žr. pav. 7). Tam, kad taškai būtų lengviau matomi, *x* ir *f(x)* ašys reprezentuojamos logaritmiška skale.



pav. 7 Tikslo funkcijos ir bandymo taškų grafikas